

Lab No 2: Implementation and Analysis of Sorting Algorithms – Merge Sort, Quick Sort (Unit 2)

1. Objective and Real-World Application

Objective: To implement and compare the Divide-and-Conquer paradigms behind Merge Sort and Quick Sort, evaluating their efficiency on random datasets.

Real-World Application: Merge Sort is heavily used for sorting massive files that don't fit into RAM (external sorting), while Quick Sort is used in highly responsive system libraries (like Java's primitive sorting or C++ `std::sort`) due to its excellent cache locality.

2. Pre-requisites (Micro-topics)

- Recursion and Call-Stack mechanics.
- Divide and Conquer algorithmic framework.
- Partitioning strategies (Lomuto vs. Hoare schemes) and Merging two sorted subarrays.

3. Brief Description and Tools

Step 1: For Merge Sort, recursively split the array into halves, then merge the sorted halves back using an auxiliary array.

Step 2: For Quick Sort, pick a pivot element, rearrange the array such that elements smaller than the pivot go left and larger elements go right, and recursively sort the partitions.

Tools: Python language only. Editors like VS Code, PyCharm, and Google Colab.

4. Descriptive Questions

Level 1: Understanding and Implementation

Question: Formulate the recursive division logic of Merge Sort and write down the algorithm/code snippet used to combine two separate sorted sub-arrays into a single sorted arrangement.

Level 2: Applying Concepts

Question: Using a pivot strategy (e.g., picking the last element), trace the partition step of Quick Sort on the array [40, 20, 10, 80, 50] and display the array boundaries at the next recursive depth.

Level 3: Advanced Challenge

Question: Discuss the structural factors that cause Quick Sort's worst-case complexity to degrade to $O(n^2)$. How does introducing a randomized pivot selection effectively safeguard the runtime against adversarial data inputs?